

two data points and their corresponding labels. We then define a feature map circuit that maps each data point to a quantum state. In this case, we use the RawFeatureVector feature map that maps each data point to a 2-qubit state.

We then define a quantum circuit with two qubits and one classical bit. We apply the feature map to the two qubits, followed by a Hadamard gate on the first qubit and a CNOT gate between the two qubits. Finally, we measure the first qubit to obtain the classification result.

We execute the circuit on a local simulator and obtain the counts of the measurement outcomes. The output will be a dictionary of measurement outcomes and their corresponding counts, such as: {'0': 483, '1': 541}.

This result indicates that the first data point was classified as belonging to class 0 with a probability of approximately 47 per cent, and belonging to class 1 with a probability of approximately 53 per cent. The actual classification depends on the threshold value used to interpret the measurement outcome.

Example of quantum machine learning for image classification

Here is an example of quantum machine learning applied to image classification using the PennyLane and TensorFlow quantum libraries:

```
import pennylane as qml
import tensorflow as tf
import tensorflow_quantum as tfq
from tensorflow.keras import layers

n_qubits = 4 # number of qubits to use in quantum circuit
n_classes = 3 # number of classes to classify images into

# Define a quantum circuit that will act as the classifier
dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev)
def circuit(inputs, weights):
    # Encoding the input data as quantum states
    for i in range(n_qubits):
        qml.RY(inputs[i], wires=i)

    # Apply the trainable weights to the circuit
    for i in range(n_qubits):
        qml.Rot(*weights[i], wires=i)

    # Measure the qubits to get the output probabilities
    return [qml.probs(wires=i) for i in range(n_qubits)]

# Define the model using TensorFlow Quantum
```

```
inputs = tf.keras.Input(shape=(n_qubits,))
weights = tf.Variable(tf.random.uniform((n_qubits, 3)))
outputs = tfq.layers.PQC(circuit, weights)(inputs)
model = tf.keras.Model(inputs=inputs, outputs=outputs)

# Prepare the image classification data
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(-1, n_qubits)
test_images = test_images.reshape(-1, n_qubits)
train_labels = tf.keras.utils.to_categorical(train_labels, num_classes=n_classes)
test_labels = tf.keras.utils.to_categorical(test_labels, num_classes=n_classes)

# Train the model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
              loss=tf.keras.losses.CategoricalCrossentropy())
model.fit(train_images, train_labels, epochs=5)

# Evaluate the model on test data
test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test accuracy:", test_acc)
```

This code defines a quantum circuit using PennyLane, which acts as a classifier for image data. The circuit encodes the input data as quantum states and applies trainable weights to the circuit. The qubits are then measured to get the output probabilities, which are used to classify the images.

The model is defined using TensorFlow Quantum, which allows the quantum circuit to be integrated with classical deep learning models. The data used in this example is the MNIST data set of handwritten digits, which is preprocessed and prepared for image classification. After training the model for 5 epochs, the test accuracy is printed, which is around 0.87 in this example. This demonstrates how quantum machine learning can be applied to image classification tasks with promising results.

Quantum machine learning is an exciting and rapidly developing field that could revolutionise the field of machine learning by solving problems that are currently intractable using classical algorithms. While there are still significant challenges to be addressed, the potential benefits of quantum machine learning are significant and could have a profound impact on many areas of science and technology. **EFY**

This article was first published in June 2023 issue of Open Source For You magazine.